BrandGuard

Pre-publication brand governance that explains, suggests, and keeps humans in control

Problem Statement	Intelligent brand-governance platform (Inkloom AI)
Submission	Round 1: The Plan
Team name	BrandGuard
Participant	Krrish Yaduka (solo)
Date	24 October 2026

One-line pitch: Others check content against a PDF. BrandGuard keeps approved facts alive, tests the guidelines themselves, and only suggests fixes it has already verified.

Contents

1. Understanding of the problem
2. Users affected
3. Proposed solution
4. Complete product workflow
5. Main features
6. Technology stack
7. Team responsibilities
8. Development plan across three rounds
9. Expected impact
10. Future scope
11. Team and participant details
12. Project links and live deployments

1. Understanding of the Problem Statement

Organisations now publish content through many employees, agencies, freelancers and AI tools. The brand rules usually live in a long PDF or wiki that nobody can apply to every post. As a result, wrong claims, outdated information, inconsistent language and off-brand visuals reach the public **before anyone notices**, and damage trust.

What the brief really asks

- Review content **before** publication against approved guidelines, facts and assets.
- Detect problems in tone, terminology, messaging, visuals, accessibility and unsupported promotional claims.
- **Explain** every issue and suggest a fix that preserves the creator's purpose, instead of just rejecting.
- Support review and approval with comments, roles and revision history.
- Surface repeated mistakes and unclear guidelines, while keeping humans in control and confidential material protected.

Three root causes we identified

- **Facts expire silently.** An offer ends or a price changes, but drafts and scheduled posts keep the old version.
- **Guidelines are ambiguous.** When people read a rule differently it gets broken constantly, and no tool tells you the rule is the problem.
- **Rejection without help.** Checks say 'no' but do not preserve what the creator was trying to say.

Scope: what we solve and what we do not

We solve pre-publication review of **text and image content**, with a review workflow and governance insights. We are not building a full digital asset manager or a social media scheduler.

Evidence of the problem

Fill in with your real findings before submitting. Do not submit placeholders.

Evidence type	What to report
Interviews	[N] interviews with marketing managers, agency freelancers and a compliance reviewer. Key insight: [quote]
Survey	[N] responses. [x%] said guidelines are checked manually or not at all
Guideline audit	[3-5] public brand guidelines reviewed. [N] vague rules found, e.g. 'friendly yet professional'
Public incidents	[Links to brands that retracted content over false claims or tone]

2. Users Affected by the Problem

User	Need	How the problem hurts them
Creator (employee, agency, freelancer, AI-tool user)	Fast, clear feedback with usable fixes	Does not know the rules, gets vague rejections, reworks repeatedly
Brand / marketing manager	Consistency at scale, visibility into patterns	Becomes the bottleneck, reviews everything manually, still misses errors
Reviewer (legal / compliance)	See only risky items, with sources	Flooded with low-risk items, cannot focus on regulated claims
Admin	Control of data, roles and AI use	No clean way to manage guidelines, facts and access

User	Need	How the problem hurts them
The organisation	Trust and reputation	A wrong price or false claim going live damages credibility

Organisational setting: a mid-sized organisation with roughly 10 to 200 content producers and at least one external agency, publishing across social, web, email and documents.

3. Proposed Solution

BrandGuard is a pre-publication reviewer that checks content against the organisation's own approved material, explains each finding with its source, and suggests verified fixes. Humans make every final decision.

The core principle

Every finding must cite a retrieved guideline clause or approved fact. If no source exists, BrandGuard says **'no rule found, human decision needed'** instead of guessing. This makes results reliable and explainable.

What makes it different

Typical approach	BrandGuard
Checks only new content against a PDF	**Fact expiry with reverse alerts:** approved facts have owners and expiry dates. When a price or offer changes, every draft and scheduled item now wrong is found and its owner alerted
Counts how often rules are broken	**Guideline testing:** generates edge cases for each rule and checks where AI, manager and reviewer disagree. Disagreement flags an ambiguous rule, with a clarity score and suggested rewording for human review
Suggests a rewrite that may break another rule	**Verified suggestions:** extracts the creator's intent, then re-runs the full checker on each candidate fix and shows only those that pass
Catches problems at the end	**Prevention at source:** an MCP server and API let AI writing tools fetch rules before writing and self-check afterwards

Example finding

Content	Finding	Suggested fix
'We deliver the fastest in India.'	No approved proof exists. Source: Claims Policy 4.2 (superlatives need cited evidence). Severity: High. Routed to reviewer	'Delivery in 2 days across 500+ cities' (matches approved fact F-12, valid till 30 Nov)

4. Complete Product Workflow

#	Stage	What happens
1	**Setup (Admin, once)**	Upload guidelines, approved facts and claims (each with owner and expiry date), terminology glossary and brand assets (logo, colours, fonts).
2	**Submit (Creator)**	Paste text or upload an image. AI tools can submit through the API or MCP connection.
3	**Review**	Deterministic checks (glossary, contrast, alt text) run in code. Retrieval-grounded AI checks cover tone, claims and facts.
4	**Explain**	Each finding shows the exact span, severity, reason, and the rule or approved source cited.
5	**Fix**	Creator sees 1 to 2 verified rewrites and can accept, edit or dispute. Nothing is applied automatically.

#	Stage	What happens
6	**Route and approve**	Low-risk content goes to the manager. Unsupported or regulated claims and low-confidence findings go to an authorised reviewer. Comments and every version are tracked.
7	**Receipt**	Each approval stores content version, rule versions, sources cited, AI service used, what data was sent out, and who approved.
8	**Insights**	Dashboard shows recurring mistakes by team or agency and which guidelines look ambiguous.
9	**Reverse alert (continuous)**	When an approved fact changes or expires, affected drafts and scheduled content are flagged and owners alerted.

5. Main Features We Intend to Build

Committed MVP

- **Knowledge base:** ingest guidelines, approved facts with owners and expiry, terminology glossary, brand assets (demo brand, about 10 to 15 rules).
- **Text and image review:** tone, terminology, facts, unsupported claims, visual consistency, accessibility.
- **Grounded, explainable findings:** every issue cites a source or is marked 'human decision needed'.
- **Verified suggestions:** intent extraction, candidate fixes, re-check, then show.
- **Fact expiry with reverse alerts** on a sample content set.
- **Review workflow:** creator, manager and reviewer roles, comments, approvals, version history.
- **Audit receipts** with AI-use disclosure for every approval.
- **Insights:** top recurring violations and a clarity score for the demo rules.

Roadmap, shown as design only

- PDF and slide ingestion
- Google Docs plugin
- Slack notifications
- Multi-brand management

6. Technology Stack

Layer	Choice	Why
Frontend	Next.js + Tailwind	Fast to build review UI with inline highlights and comment threads
Backend	FastAPI (Python)	Strong ML and LLM ecosystem, quick structured APIs
Database	Postgres + pgvector	One database for roles, versions, audit and vector retrieval keeps the system simple
LLM	Claude with structured JSON output	Rule-grounded reasoning; outputs forced into a schema we validate
Deterministic checks	Plain code	Glossary matching, WCAG contrast, alt-text presence must be exact, not AI opinion
Vision	Multimodal model + code-based colour extraction	Model describes the image; palette and contrast are measured in code
Integration	MCP server + REST API	Lets AI writing tools fetch rules first and self-check after

Layer	Choice	Why
Access and privacy	Role-based access, per-organisation isolation, encryption at rest	Matches roles and protects confidential material

Hardest technical problem and our answer

LLMs invent rules, miss conflicts, and write rewrites that fix one issue while breaking another. Our architecture:

- **Grounded retrieval:** only relevant clauses and valid, unexpired facts are retrieved. A validator rejects any finding without a real citation ID.
- **Intent extraction** before suggesting fixes.
- **Verify-then-suggest loop:** every candidate fix is re-checked against all rules and the original intent.
- **Confidence and escalation:** low-confidence or conflicting findings go to a human.

Responsible design

- **Human oversight:** AI only recommends; approval always needs a person.
- **Confidentiality:** per-organisation isolation, encryption at rest, no training on customer data, optional redaction before external calls.
- **Disclosure:** every report states the AI service used and what data was sent.
- **Auditability:** immutable decision log with the guideline version attached.
- **Known limits shown:** visual checks are less reliable than text checks and are labelled with lower confidence.

7. Responsibilities of Each Team Member

BrandGuard is a solo project (team size: 1). Krrish Yaduka is responsible for every area, organised below as work streams so the plan stays clear and prioritised.

Work stream	Responsibilities
AI and backend	Grounded retrieval engine, citation validator, verify-then-suggest loop, fact expiry and reverse alerts, MCP server and API
Frontend and workflow	Next.js review UI, roles, comments, approvals, version history, audit receipts, insights dashboard
Research, data and quality	Interviews and survey, demo brand guidelines and facts, test cases, clarity-scoring evaluation, accessibility and visual checks
Pitch and documentation	Submission documents, demo script, final presentation

Because there is one builder, scope is deliberately limited to the committed MVP in section 5 and build order is driven by risk (section 8).

8. Development Plan Across All Three Rounds

Exact round dates and requirements were not stated in the brief we have. The plan below is organised by what each round should deliver; confirm dates with the organisers.

Round	Goal	Deliverables
Round 1: The Plan	Prove the problem and the approach	This document; evidence from interviews, survey and guideline audit; architecture and workflow design; risk plan

Round	Goal	Deliverables
Round 2: Working prototype	Show the core engine works	Knowledge-base ingestion; grounded findings with citation validation; text review end to end; basic review UI with roles; fact expiry demo
Round 3: Final product	Demonstrate value on realistic workflows	Image review; verify-then-suggest loop; comments, approvals and version history; audit receipts; insights and clarity score; MCP and API demo; polished demo with a realistic content set

Build order, prioritised by risk

- 1. Grounded-findings engine with citation validation (hardest, de-risk first).
- 2. Review UI and role workflow.
- 3. Fact expiry and reverse alerts.
- 4. Verify-then-suggest loop.
- 5. Insights and clarity scoring.
- 6. MCP and API endpoint, then demo polish.

Risks and mitigations

Risk	Mitigation
Clarity scoring is hard to do well	Demonstrate on about 10 rules with simulated reviewer disagreement; describe it as flagging likely-ambiguous rules for human review
Image checks less reliable than text	Measure palette and contrast in code; label visual findings with lower confidence
Hallucinated findings	Citation validator rejects anything without a real source
Scope creep	Fixed MVP list; everything else is roadmap

9. Expected Impact of the Solution

These are the outcomes we will measure in the demo, not claims of results already achieved.

For	Expected impact	How we will measure it
Creators	Fewer review rounds; clear reasons and usable fixes	Review rounds per item before and after, on the sample set
Managers	Less manual checking; focus on exceptions	Share of items auto-cleared versus escalated
Reviewers	Only risky and unsupported claims reach them	Number and share of items routed to reviewers
Organisation	Fewer wrong or outdated claims published; auditable decisions	Stale-fact items caught by reverse alerts; findings with valid citations
Brand owners	Clearer guidelines over time	Rules flagged ambiguous and later reworded

10. Future Scope After the Event

- **More formats:** PDF, slide decks, video captions and audio scripts.
- **Plugins:** Google Docs, Microsoft Word, Figma, and CMS integrations to check content where it is written.
- **Notifications and tools:** Slack and Teams alerts, and ticketing integration.
- **Multi-brand and multi-language:** agencies managing several brands, and regional-language tone and terminology checks.

- **Regulated industries:** configurable policy packs for finance, health and advertising rules, always with human sign-off.
- **Deeper governance analytics:** trends by agency, team and channel; automatic proposals to improve guidelines.
- **Enterprise readiness:** single sign-on, self-hosted or private-model deployment for sensitive materials, and compliance reporting.
- **Business model:** subscription per organisation by number of reviewers and content volume.

11. Complete Team and Participant Details

Field	Details
Team name	BrandGuard
Team size	1 (solo participant)
Full name	Krrish Yaduka
Role	Sole builder: AI, backend, frontend, research and pitch
Institution	Adamas University, Kolkata
Programme and year	B.Tech CSE (AI and ML), final year
GitHub	krrish2803
Relevant experience	Machine Learning Intern, Unified Mentor; AI/ML Backend Developer Intern, Convico.AI Nexus Labs

12. Project Links & Live Deployments

All core components of BrandGuard are deployed live, operational, and open-source:

Deliverable / Component	Live URL & Access	Status & Details
Live Platform & Dashboard	https://brandguard-vy9i.onrender.com/#/dashboard	Operational (Render Cloud)
Product Landing Page	https://brand-1guard.netlify.app	Published (Netlify CDN)
GitHub Repository	https://github.com/krrish2803/BrandGuard	Public (MIT, 116 Tests Passing)
Backend REST API & Health	https://brandguard-vy9i.onrender.com/api/health	Active (200 OK, Dual Storage)

Note: All URLs above are interactive hyperlinks. The platform runs hermetically and connects live with NVIDIA NIM.